Listing 8-2. *Flicker Example Code*

```
int ledPin = 11;
int flicker[] = {64, 22, 4, 28, 6, 130, 186, 120};
int i = 0;

void setup() {
  randomSeed(analogRead(A5));
}

void loop() {
  analogWrite(ledPin, flicker[i]);
  delay(random(100, 500));
  i++;
  if (i == (sizeof(flicker)/2)) i = 0;
}
```

This short but compact example sketch will flicker an LED at varying brightness levels as set by data stored in the `flicker[]` array with a delay between each value. Using the random functions discussed earlier in this chapter, this delay will be randomly determined to be between a tenth to one half of a second in length. The array `flicker[]` contains eight elements with values between 0 and 255 previously collected from wind speed data. More data would create a better effect but we kept it small for this example. Using actual data in this case instead of randomly generated numbers is essential to creating a more lifelike behavior that makes our flickering LED even more candle-like.

Inside the `loop()` function, we write the value of each element to the `ledPin`, in this case pin 11, beginning with the first element at index 0. By using the `random()` function inside the `delay()` function, we create a random pause of varying lengths to make things a little more unpredictable. Each time through `loop()`, the variable `i` is incremented by one to advance the array index one step moving to the next brightness level.

So our last line in this sketch will perform a simple test to check if our counter `i` has reached the end of our array, whatever size it might be, and if so it will reset our counter to the value 0. In this way, we start with the value at the beginning of the list, which is 64, and continue each time through the loop until we hit the last value at index 7, which in this case is 120, before starting all over again with the first value. This is pretty useful so we can add values to our array as much as we want either manually through adding extra values in our code, or through storing a multitude of sensor readings in the loop.

Array Limits and sizeof()

We should probably back up a bit here and explain a little more about the limits of an array. It is really important that we are careful to not access an array beyond the number of elements declared in the array's dimension, whether explicitly declared by us in the declaration of the array or implicitly declared by the Arduino compiler counting the values assigned to the array. For example, in Listing 8-2 if we tried to access a ninth element of the `flicker[]` array at index 8, which does not exist, we would be reading from parts of the program memory that might be in use by other things. Needless to say, this can cause weird things to happen. Just remember that Arduino C has no way to cross-check our array declarations and would allow us to exceed the boundaries of our arrays. It becomes our responsibility to make sure that any calls to an array stay within its dimension.

To be sure we did not exceed the limits of the array's dimension in Listing 8-2, we needed a way at the end of the main loop to know when we reached the end of the array. To do this, we brought in a nice little utility function in the Arduino library called `sizeof()`. This function will return the total number of